

How this product thinks — answers to the 5 questions

Velocity asked to see how I think, not a finished build. This maps each of their five questions to a clear answer, the reasoning behind it, and exactly what to put on a slide. Everything here is made tangible in the prototype + the animated architecture diagram.

The spine of every answer: the prototype is a **Customer Context Layer** — it sits between scattered source data and the AI agents, and answers *"what does THIS agent need to know about THIS customer to do THIS job, right now."* Each agent declares a small **Context Contract**; the layer resolves identity, derives traits, and serves only the fields that contract needs.

Q1 — How to unify & organize the data so it's clean, reliable, AI-usable

Answer: Three layers, in order. **(1) Identity resolution** turns many scattered records into one customer. **(2) An event store** keeps every raw fact immutably. **(3) A trait store** holds derived, AI-ready signals recomputed from those events.

Reasoning (why this order, why it's clean):

- **Identity is the keystone** — nothing downstream is trustworthy until "4 records = 1 human." We resolve with escalating signals, each carrying a *confidence*: deterministic (phone/email) → probabilistic (name + address + device, AI-scored) → behavioral (anonymous session → known buyer) → cross-brand graph join. We tune for **precision over recall**: a wrong merge is worse than a missed one, so low-confidence links are held for review, never silently merged.
- **Two stores, on purpose.** The event store is the immutable source of truth (orders, chats, shipments — append-only, replayable). The trait store is *derived* (RFM, LTV, RTO-risk, churn, sentiment) and **recomputed from events, never hand-set**. This is what makes it *reliable*: a trait looks wrong? Replay the events and recompute — no mystery state, fully auditable.
- **AI-usable** means agents read **traits**, not raw tables. So we can change how a trait is computed without breaking a single agent.

Put on a slide: the architecture diagram — *Sources → Identity Resolution (keystone) → Event store + Trait store → Context API*. One line under it: *"Clean = identity-resolved with confidence. Reliable = derived from an immutable event log. AI-usable = agents read traits, not tables."* **Show in the demo:** click **Priya** → 4 scattered records (storefront, WhatsApp, web session) collapse into one profile with anchor + method + confidence.

Q2 — Where AI does the work, instead of rules or humans

Answer: AI replaces manual effort at **five points** in the pipeline — and the prototype proves the last two *live*.

Where	Today (manual / rules)	With AI
Ingestion	Engineers hand-map each brand's schema	LLM maps any source's fields → canonical events → onboard a brand in hours, not a sprint
Identity	Exact-match rules only; the rest is dropped	Probabilistic matcher scores fuzzy links with a confidence
Traits	Analysts write SQL to define segments	Model extracts sentiment, intent, anxiety, RTO-risk from raw events
Segmentation	Marketer writes SQL / waits on the data team	Plain English → live segment (<i>shown</i>)
Action	Static templated copy	Per-customer message generated from the task bundle (<i>shown</i>)

Reasoning: the senior point isn't "we call an LLM." It's that AI removes the **human bottleneck** at each step where the status quo needs an engineer, an analyst, or a rules file. The forcing-function method — *let the agents' needs define the data* — is itself AI-first thinking: you design backward from the decision, not forward from a schema.

Put on a slide: the table above, titled "*AI does the work — 5 places we delete manual effort.*" Star the two the prototype shows live. **Show in the demo:** type "*risky COD orders I shouldn't ship*" → AI turns English into a structured segment (no SQL). Then **Generate live** → Claude writes a real WhatsApp message from only the task-scoped fields.

Q3 — New AI features, including ones brands don't have today

Answer: On this layer, today's 6 agents are just contracts. The genuinely *new* capability is three things no single brand can do alone:

1. **Anticipatory, not reactive** — WIMO that messages *before* the customer asks, because we see "in transit + first order + 2 prior anxious chats."
2. **Self-tuning agents** — every send + outcome (converted? returned? replied?) writes back as an event → traits like `past_nudge_response` sharpen → the *next* decision is better. The layer **learns**; no analyst in the loop.
3. **The cross-brand moat** — a customer returns one order at Brand A: annoying. We see **4 of 5 returned across three brands**: a pattern. We pause COD *before shipping at a loss* — at a brand they've never bought from. Every new brand makes the RTO/churn priors better for every other brand.

Reasoning: the assignment's first paragraph says "*run as one platform serving many brands.*" Most people treat multi-brand as a scaling headache. Reframing it as the **moat** — pooled, identity-resolved signals — is the product-strategy insight. It's also defensible: it compounds, and a single-brand competitor structurally cannot replicate it.

Put on a slide: "*What brands can't do today*" with three rows — Anticipatory · Self-tuning · Cross-brand moat. Lead the room with the Vikram story. **Show in the demo:** click **Vikram** → the cross-brand return signal (4/5 across 3 brands) → run **RTO Shield** → "COD paused — network signal from 3 brands." (*In the new dataset there are several MOAT customers, so the segment "risky COD" lights up a whole cohort.*)

Q4 — How it scales across brands, products, and growing data

Answer: The **Context Contract** is the whole trick. A new product = a new contract over existing traits = **no re-architecture**. Platform engineering cost stays $\approx O(1)$ in the number of products.

Reasoning:

- A contract is a small declarative list of trait/event keys + decision logic. It reads the *existing* trait store — usually needs zero new data. When it needs a new trait, that trait is computed once from events and is then available to *every future* agent.
- **This is the answer to "products you haven't built yet":** I added a 7th agent — **Returns & Refunds** — as a pure contract over traits that already existed. No pipeline, no schema migration. That's the proof.
- **Multi-tenancy & governance** (the unglamorous part that decides if it's real): brand-isolated by default; cross-brand sharing is a *governed* join that shares **derived scores, never raw PII**; DPDP-aware; auditable because traits recompute from the log.
- **Growing data doesn't break it** because ingestion is AI-mapped (new sources → canonical events) and traits are recomputed, not migrated.

Put on a slide: New product? → Write a contract. → Ship. (*not: design a schema, build a pipeline, re-resolve identity.*) Sub-line: "Cost is $O(1)$ per product — the roadmap speeds up as it grows." **Show in the demo:** keep the same customer, switch the **task** — the contract and the served context change. Then point out the **Returns & Refunds** agent: "this one didn't exist last week; it's just a new contract."

Q5 — What I'd build first, and what I'd defer (and why)

Answer: Prioritized by **value × data-readiness × reversibility-of-risk**. Build where the data is already clean and a wrong action is cheap; defer where it isn't.

Build first

- **Phase 0 — Foundation:** identity resolution + event store + trait store + Context API. *Nothing works without it; highest leverage, lowest glamour. No agent ships until "4 records → 1 human" is trustworthy.*
- **Phase 1 — low-regret, clean-data agents:** Proactive WIMO (data already clean, worst case is a friendly extra message), Cart Recovery (strong live signal, reversible, fast revenue), COD→Prepaid (direct margin, low-risk nudge).

Defer (and why)

- **RTO Shield / cross-brand** — highest revenue protection, but pausing COD on a *good* customer is a costly false positive; ship once the model + cross-brand graph have volume to trust.
- **Returns/refunds adjudication, autonomous send, broad broadcasts, ML-retraining infra** — money/policy risk, high blast radius, or low differentiation. Earn trust first; autonomy is a dial, not a launch toggle.

Reasoning — the restraint is the signal: the prototype itself has **no database, no auth, no real integrations — on purpose**. Knowing what *not* to build in a 1–2 day window is the same judgment the roadmap question tests. Saying "I'd defer returns adjudication and here's why" is a stronger answer than building all ten badly.

Put on a slide: a 2-column "Build first / Defer" with the one-line *why* on each. End with: "*What not to build is a product decision, not an oversight.*"

The one line to repeat

A CDP answers *who is my customer*. We built the layer that answers *what does THIS agent need to know about THIS customer to do THIS job, right now* — clean because it's identity-resolved and event-sourced, AI-first because AI does the work at every step, extensible because a new agent is just a new contract, and defensible because across many brands the data compounds into a moat no single brand can build.